01_vector_addition

Visual guide — Vector Addition

This note complements the [official Triton tutorial](https://triton-lang.org/main/getting-started/tutorials/01-vector-add.html). Read it before diving into code.

Goal

Your first Triton kernel: SPMD programs, block pointers, masks, and benchmarking against PyTorch.

Mental model

```mermaid
flowchart LR
  subgraph grid["Launch grid (1D)"]
    P0["Program 0\noffsets 0..BLOCK-1"]
    P1["Program 1\noffsets BLOCK..2BLOCK-1"]
    Pn["Program N\n..."]
  end
  DRAM[(Global memory\nx, y, out)]
  P0 --> DRAM
  P1 --> DRAM
  Pn --> DRAM
```

Official benchmark figure



Vector-add throughput vs problem size (official Triton tutorial benchmark).

Step-by-step (what each phase does)

1. **Host setup** — Allocate tensors on `DEVICE`, pick dtypes, and define launch grid meta-parameters (`BLOCK_SIZE`, tile sizes, `num_warps`).
2. **Kernel** — Each program instance loads a tile, computes in fast memory (SRAM/shared), stores results. Use **masks** when dimensions are not multiples of the tile.
3. **Correctness** — Compare to PyTorch / reference with `torch.allclose` on odd sizes (tests padding paths).
4. **Benchmark** — Use `triton.testing.do_bench` and `perf_report` for GB/s or TFLOPS; interpret as memory-bound vs compute-bound.

Tips

- If performance flatlines, check occupancy (registers/shared mem) and whether the kernel is **memory-bound** (fusion helps) or **compute-bound** (tile sizes / autotune matter).
- Re-run `python scripts/download_tutorial_images.py` if figures are missing offline.

See also

- [Tutorials index](https://triton-lang.org/main/getting-started/tutorials/index.html)
- Repo notebook: `01_vector_addition.ipynb`

benchmark-vector-add.png

